TEAM 4 – Ctrl+Alt+Deploy

DOCKER – SESSION REPORT

Container Lifecycle, Port Mapping, Nginx & Reverse Proxy

Session Date: 06 November 2026

Platform: AWS EC2

Operating System: Ubuntu

Attendance and Team Roles

Team Member	Role	Contribution
Dinesh Konda	Lead	Managed team activities
Katragadda Maheeja	Driver	Executed practical commands
Jampani Jahnavi	Navigator	Verified commands and configurations
Karavadi Pavan Kumar	Breaker	Tested errors and troubleshooting
Hema Sri Janga	Scribe	Prepared documentation and screenshots
Ajay Siddela	Presenter	Presented the final implementation

1. Introduction

In this session, we explored Docker container lifecycle management, port mapping, Nginx installation and reverse proxy configuration.

We used an Ubuntu EC2 instance on AWS to deploy multiple Docker containers. We then configured Nginx to act as a reverse proxy and route incoming browser requests to five different backend applications using different ports.

2. Objectives

The main objectives of this session were:

1. Understand the Docker container lifecycle.
2. Practice creating, starting, stopping, restarting, pausing and removing containers.
3. Understand Docker port mapping.
4. Install and configure Nginx on an Ubuntu EC2 instance.
5. Run multiple containers on different host ports.
6. Configure Nginx as a reverse proxy.
7. Access multiple applications through a single public IP address.
8. Identify and troubleshoot errors during implementation.

3. Environment Setup

We used the following environment for the practical implementation.

Component	Details
Cloud Platform	AWS
Service	Amazon EC2
Operating System	Ubuntu
Container Platform	Docker
Web Server	Nginx
Backend Applications	Apache HTTPD
Public Access	EC2 Public IPv4
Configuration	Nginx reverse proxy

Step 1: Connect to the EC2 Instance

Connected to the Ubuntu EC2 instance using SSH.

Step 2: Verify Docker Installation

```
None
sudo docker --version
sudo docker ps
```

These commands helped verify that Docker was installed and accessible.

4. Practical Implementation

TASK 1: Docker Container Management

4.1 Exploring Docker Container Lifecycle

Docker containers can be managed through different commands depending on whether we need to create, execute, pause, resume, or delete them. In this task, we practised the essential commands used to control containers.

Docker Command	Description
`docker create`	Initializes a new container without executing it.
`docker start`	Activates a previously created container.
`docker stop`	Gracefully terminates a running container.
`docker restart`	Stop and start the container again.
`docker pause`	Suspends all processes inside a container.
`docker unpause`	Continues the execution of suspended processes.
`docker rm`	Deletes a container that is no longer running.

Key Observation:

The `docker run` command simplifies container creation and execution by combining the functionality of `docker create` and `docker start` into a single command.

4.2 Launching a Docker Container

We created a new Ubuntu-based container and assigned it the name `my-server`.

Command:

```
None
sudo docker run -dit --name my-server ubuntu bash
```

Command Breakdown:

- `-d` – Executes the container in detached mode.
- `-i` – Keeps the input stream available.
- `-t` – Provides a terminal interface.
- `--name my-server` – Sets a custom identifier for the container.
- `ubuntu` – Specifies the image used to build the container.
- `bash` – Starts the Bash shell inside the container.

This command allowed us to create and run the Ubuntu container in the background.

4.3 Monitoring Active Containers

To identify which Docker containers were currently executing, we used the following command.

```
None
sudo docker ps
```

Observation:

The output displays information about active containers, including their IDs, image names, execution status, assigned names, and port configurations.

4.4 Stopping Container Execution

We temporarily stopped the running container to understand how Docker handles container termination.

Command:

```
None
sudo docker stop my-server
```

Observation:

The container's processes were stopped, but its configuration and data remained available. It could be started again whenever required.

4.5 Listing All Available Containers

To inspect both active and inactive containers, we executed:

```
None
sudo docker ps -a
```

Observation:

This command provides a complete list of containers, including those that are currently running, stopped, or exited. It helped us track the container's current state.

4.6 Restarting and Activating Containers

We tested how to bring back a stopped container and restart an existing one.

Commands:

```
None
sudo docker start my-server

sudo docker restart my-server

sudo docker ps
```

Observation:

- `docker start` – Brings the stopped container back into operation.
- `docker restart` – Reinitializes the running container.
- `docker ps` – Confirms whether the container is active after these operations.

4.7 Deleting a Docker Container

Finally, we performed the container removal operation to understand how Docker handles resource cleanup.

Commands:

```
None
sudo docker stop my-server

sudo docker rm my-server

sudo docker ps -a
```

Observation:

First, we stopped the container to ensure it was not running. Then, we removed it using `docker rm`. The final command helped verify that the container was no longer listed.

Result: Successfully performed the basic Docker container management operations, including creation, execution, stopping, restarting, inspection, and removal.

Nginx Installation Steps on Ubuntu EC2 Instance

Step 1: Check Nginx Installation

```
None
nginx -v
```

Step 2: Update Package Lists

```
None
sudo apt update
```

Step 3: Install Nginx

```
None
sudo apt install nginx -y
```

Step 4: Verify Nginx Version

```
None
nginx -v
```

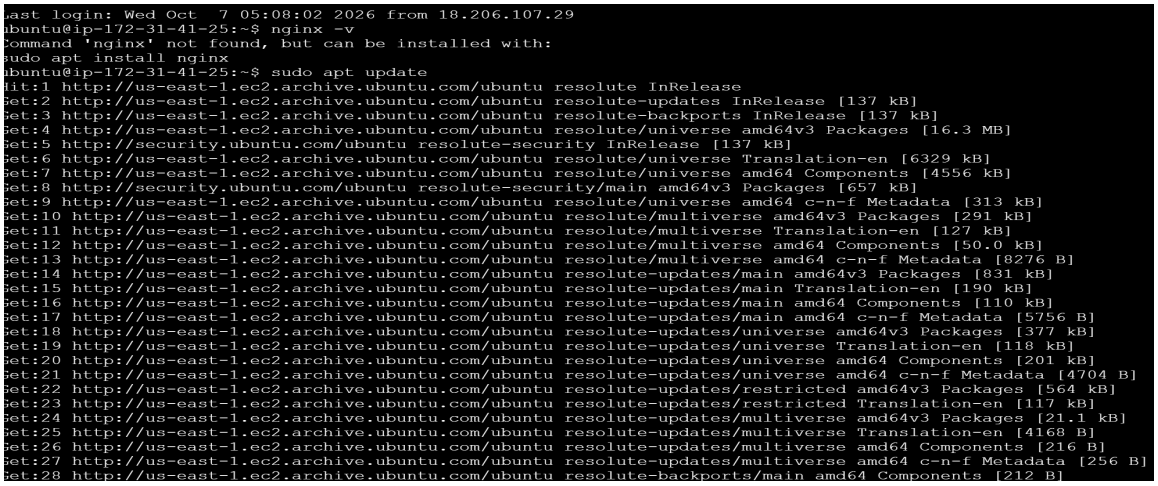
Step 5: Check Nginx Service Status

```
None
sudo systemctl status nginx
```

Step 6: Check Nginx Listening Port

```
None
sudo ss -lntp | grep nginx
```

Result: Nginx was successfully installed on the Ubuntu EC2 instance and is running on the



default HTTP port 80.

```
ubuntu@ip-172-31-41-25:~$ sudo apt install nginx -y
Installing:
  nginx

Installing dependencies:
  nginx-common

Suggested packages:
  fcgiwrap  nginx-doc  ssl-cert

Summary:
  Upgrading: 0, Installing: 2, Removing: 0, Not Upgrading: 203
  Download size: 667 kB
  Space needed: 1880 kB / 4600 MB available

Get:1 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute-updates/main amd64v3 nginx-common all 1.28.3-2ubuntu1.12 [38.2 kB]
Get:2 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute-updates/main amd64v3 nginx amd64 1.28.3-2ubuntu1.12 [629 kB]
Fetched 667 kB in 0s (17.5 MB/s)
Preconfiguring packages ...
Selecting previously unselected package nginx-common.
(Reading database ... 85303 files and directories currently installed.)
Preparing to unpack .../nginx-common_1.28.3-2ubuntu1.12_all.deb ...
Unpacking nginx-common (1.28.3-2ubuntu1.12) ...
Selecting previously unselected package nginx.
Preparing to unpack .../nginx_1.28.3-2ubuntu1.12_amd64v3.deb ...
Unpacking nginx (1.28.3-2ubuntu1.12) ...
Setting up nginx-common (1.28.3-2ubuntu1.12) ...
Created symlink '/etc/systemd/system/multi-user.target.wants/nginx.service' → '/usr/lib/systemd/system/nginx.service'.
Setting up nginx (1.28.3-2ubuntu1.12) ...
 * Upgrading binary nginx
Processing triggers for man-db (2.13.1-1build1) ...
Processing triggers for ufw (0.36.2-9build1) ...
Scanning processes...
Scanning linux images...
```

```
No VM guests are running outdated hypervisor (qemu) binaries on this host.
ubuntu@ip-172-31-41-25:~$ nginx -v
nginx version: nginx/1.28.3 (Ubuntu)
ubuntu@ip-172-31-41-25:~$ sudo systemctl status nginx
● nginx.service - A high performance web server and a reverse proxy server
     Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: enabled)
     Active: active (running) since Fri 2026-11-06 03:52:50 UTC; 35s ago
 Invocation: 057c9533d84648e0b54b3f7d582c6859
       Docs: man:nginx(8)
    Process: 1832 ExecStartPre=/usr/sbin/nginx -t -q -g daemon on; master_process on; (code=exited, status=0/SUCCESS)
    Process: 1834 ExecStart=/usr/sbin/nginx -g daemon on; master_process on; (code=exited, status=0/SUCCESS)
   Main PID: 1863 (nginx)
      Tasks: 3 (limit: 627)
     Memory: 3M (peak: 7M)
        CPU: 43ms
     CGroup: /system.slice/nginx.service
             ├─1863 "nginx: master process /usr/sbin/nginx -g daemon on; master_process on;"
             ├─1866 "nginx: worker process"
             └─1867 "nginx: worker process"

Nov 06 03:52:50 ip-172-31-41-25 systemd[1]: Starting nginx.service - A high performance web server and a reverse proxy server...
Nov 06 03:52:50 ip-172-31-41-25 systemd[1]: Started nginx.service - A high performance web server and a reverse proxy server.
ubuntu@ip-172-31-41-25:~$ sudo ss -lntp |grep nginx
LISTEN 0      511          0.0.0.0:80        0.0.0.0:*    users:(("nginx",pid=1867,fd=5),("nginx",pid=1866,fd=5),("nginx",pid=1863,fd=5))
LISTEN 0      511             [::]:80           [::]:*    users:(("nginx",pid=1867,fd=6),("nginx",pid=1866,fd=6),("nginx",pid=1863,fd=6))
ubuntu@ip-172-31-41-25:~$
```

Configure Inbound Rules in EC2 Security Group

Step 1: Login to the AWS Management Console.

Step 2: Navigate to EC2 → Instances.

Step 3: Select your running EC2 instance.

Step 4: Go to the Security tab.

Step 5: Click on the associated Security Group.

Step 6: Select Inbound rules → Edit inbound rules.

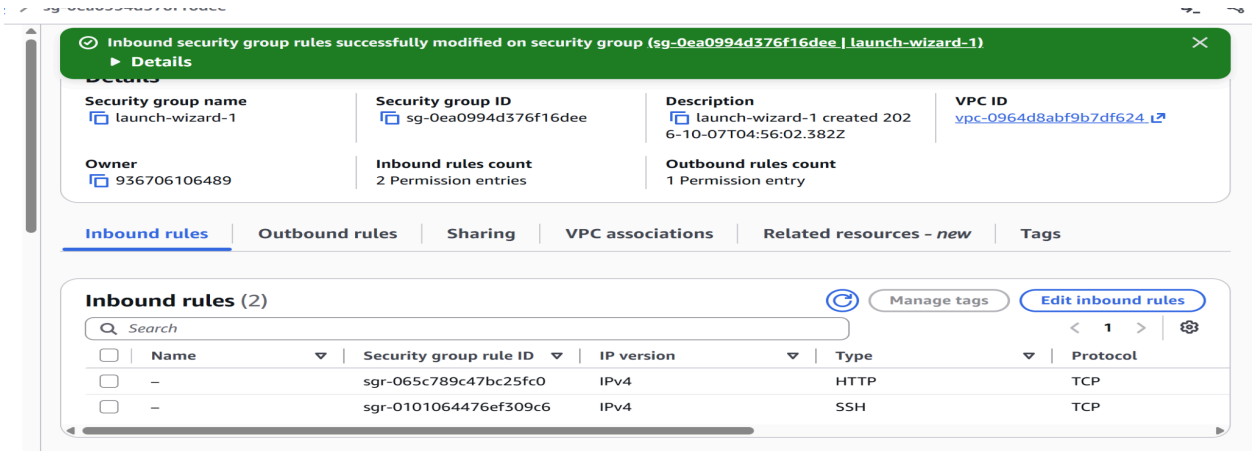
Step 7: Click on Add rule and configure the following details:

Field	Value
Type	HTTP
Protocol	TCP
Port range	80
Source	0.0.0.0/0

Step 8: Click Save rules.

Step 9: Open your browser and enter your EC2 instance's Public IPv4 address:

http://52.87.198.74





TASK 2 : Port mapping

Successfully pulled the official Nginx Docker image from Docker Hub and created an Nginx container named `nginx-container`. Configured port mapping from host port `8080` to container port `80` to access the Nginx web server through a browser.

Commands Used:

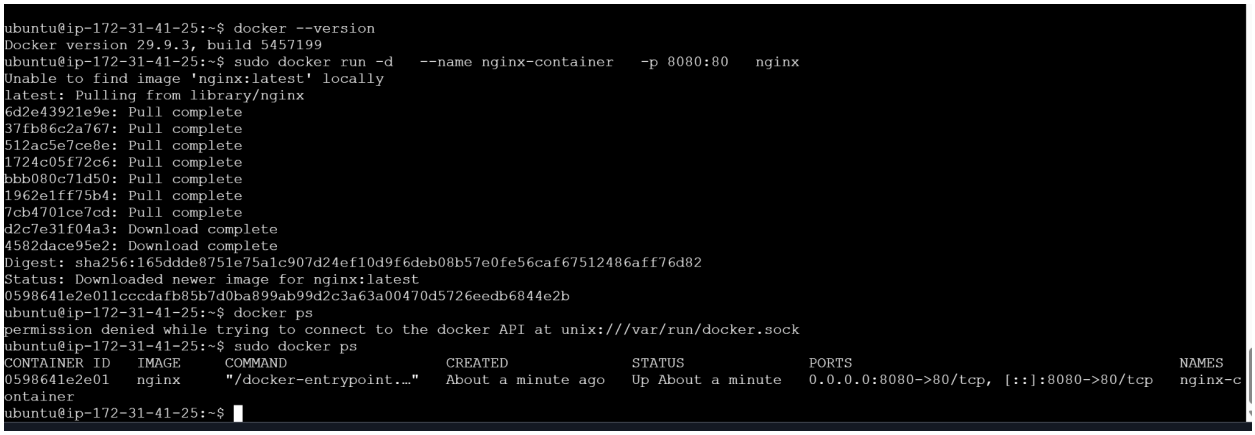
None

```
sudo docker run -d --name nginx-container -p 8080:80 nginx

sudo docker ps

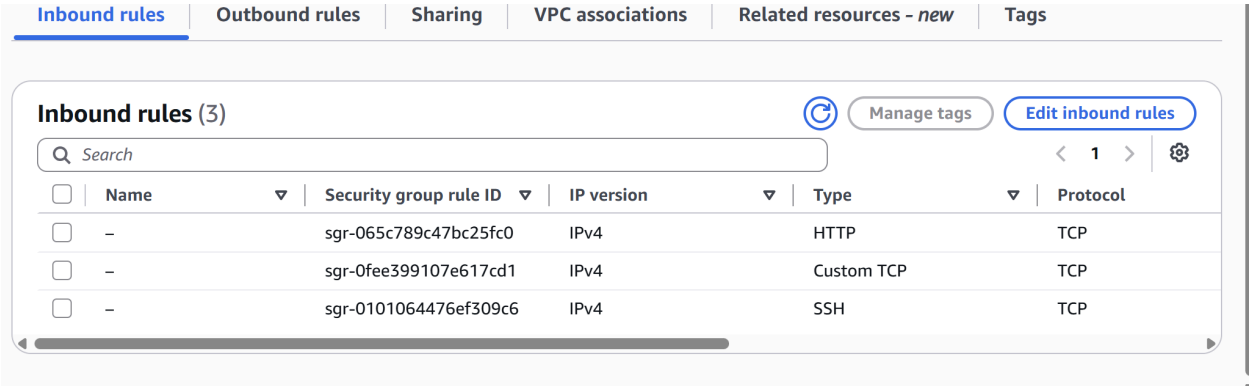
sudo docker logs nginx-container

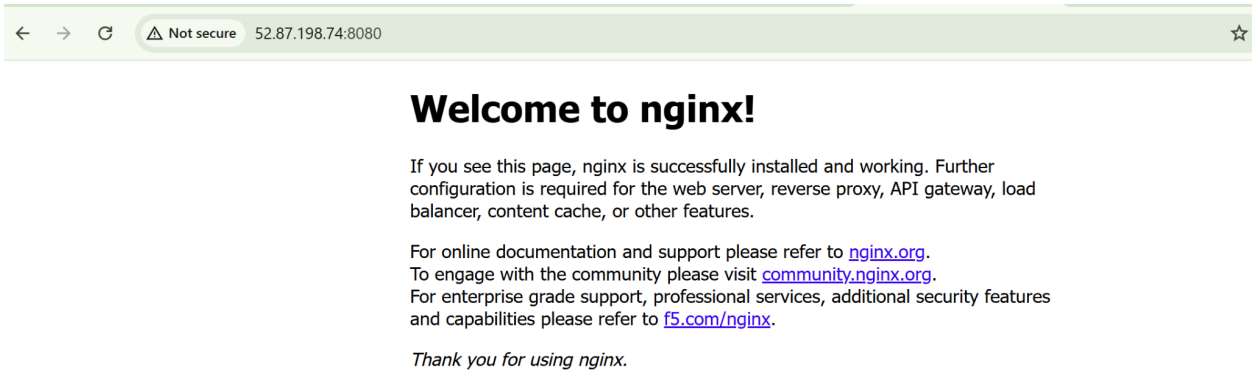
curl http://localhost:8080
```



Configured an inbound security group rule in the AWS EC2 instance to allow Custom TCP traffic on port `8080` with the source restricted to My IP. Accessed the Nginx Docker container through the EC2 Public IPv4 address using port `8080` and verified the deployment by checking the Welcome to nginx! default page in the web browser.

http://52.87.198.74 :8080



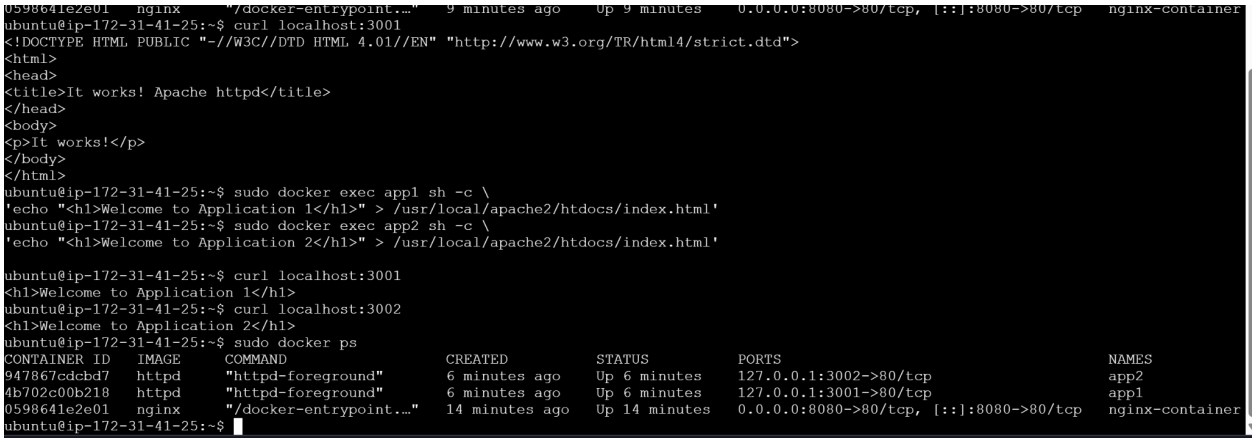


Task 3: Configuring Nginx Reverse Proxy for Five Docker Applications

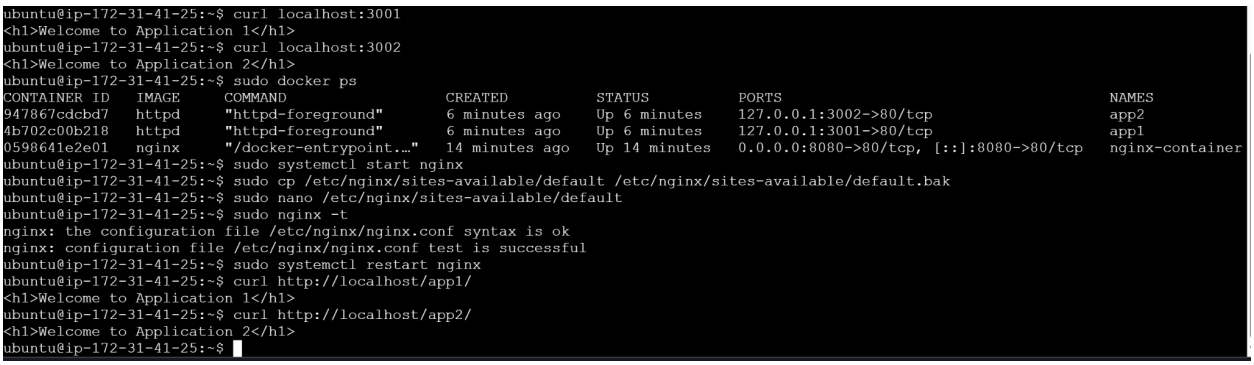
Successfully created and started two backend application containers (`app1` and `app2`) using the official Apache HTTP Server (`httpd`) Docker image. Configured port mapping to forward host port `3001` to container port `80` for app1 and host port `3002` to container port `80` for app2. Both applications are bound to `127.0.0.1`, ensuring they are accessible only through localhost and cannot be accessed directly from the internet. Verified the running containers using the `sudo docker ps` command.

```
ubuntu@ip-172-31-41-25:~$ sudo docker run -d \
  --name app1 \
  -p 127.0.0.1:3001:80 \
  httpd
Unable to find image 'httpd:latest' locally
latest: Pulling from library/httpd
5b601207ebe7: Pull complete
37fb86c2a767: Already exists
b98c7dfab78b: Pull complete
6eb2bb831c99: Pull complete
4f4fb700ef54: Pull complete
e661e3574f8f: Pull complete
134379845de0: Download complete
193260a50641: Download complete
Digest: sha256:f6a86043744215e4be90cf8804375d90022a711d892c9aa2d7085c089ee2cb54
Status: Downloaded newer image for httpd:latest
4b702c00b2180db0e4fa42b2d68ce995566bf3449f1463432cbccc019e6d4ba1
ubuntu@ip-172-31-41-25:~$ sudo docker run -d \
  --name app2 \
  -p 127.0.0.1:3002:80 \
  httpd
947867cdcbd72d22f750bba297d5b3e73bd3dac3490aa56c64b3a4cbe8477456
ubuntu@ip-172-31-41-25:~$ sudo docker ps
CONTAINER ID   IMAGE     COMMAND              CREATED          STATUS          PORTS                                   NAMES
947867cdcbd7   httpd     "httpd-foreground"   16 seconds ago   Up 15 seconds   127.0.0.1:3002->80/tcp                  app2
4b702c00b218   httpd     "httpd-foreground"   31 seconds ago   Up 31 seconds   127.0.0.1:3001->80/tcp                  app1
0598641e2e01   nginx     "/docker-entrypoint.…"   9 minutes ago    Up 9 minutes    0.0.0.0:8080->80/tcp, [::]:8080->80/tcp   nginx-container
ubuntu@ip-172-31-41-25:~$
```

Successfully configured two separate Apache backend applications using Docker. Created individual HTML welcome pages for Application 1 and Application 2 by executing commands inside their respective containers. Verified both applications using curl on localhost ports 3001 and 3002.



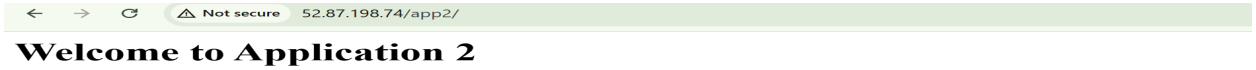
Successfully started the host Nginx service and configured Nginx as a reverse proxy by modifying the default configuration file. Created a backup of the original configuration and configured separate routing rules for two backend applications. Application 1 is mapped to `127.0.0.1:3001` through `/app1/`, and Application 2 is mapped to `127.0.0.1:3002` through `/app2/`. This configuration allows Nginx to forward incoming HTTP requests to the respective Docker containers based on the requested URL path.



Application 1: `http://52.87.198.74/app1/`



Application 2: `http://52.87.198.74/app2/`



Issue: Nginx Service Restart Failed

1. Problem:

While restarting the Nginx service, the following error occurred:

```
None
Job for nginx.service failed because the control process exited
with error code.

See "systemctl status nginx.service" and "journalctl -xeu
nginx.service" for details.
```

2. Reason:

Although the Nginx configuration syntax test was successful, the Nginx service failed to restart due to a service startup issue.

3. Troubleshooting Steps:

First, checked the Nginx service status:

```
None
sudo systemctl status nginx
```

Then, checked the detailed error logs:

```
None
sudo journalctl -xeu nginx.service
```

Checked whether port 80 was already being used by another process:

```
None
sudo ss -lntp | grep ':80'
```

Identified the cause of the service failure and corrected the issue.

4. Solution:

After correcting the identified issue, tested the Nginx configuration:

```
None
sudo nginx -t
```

Restarted the Nginx service:

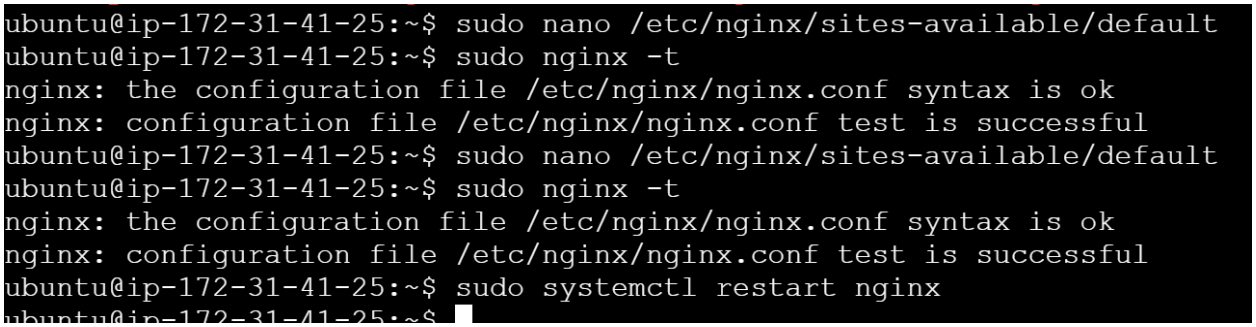
```
None
sudo systemctl restart nginx
```

Verified the service status:

```
None
sudo systemctl status nginx
```

5. Result:

Successfully restarted the Nginx service and verified that it was running without errors.



1. Check Existing Docker Containers

First, check the existing Docker containers (`app1` and `app2`).

```
None
sudo docker ps
```

2. Create the Remaining Three Apache Containers

Created three additional Apache HTTPD containers on different localhost ports.

Application 3:

```
None
sudo docker run -d --name app3 -p 127.0.0.1:3003:80 httpd
```

Application 4:

```
None
sudo docker run -d --name app4 -p 127.0.0.1:3004:80 httpd
```

Application 5:

```
None
sudo docker run -d --name app5 -p 127.0.0.1:3005:80 httpd
```

3. Configure Separate Welcome Pages

Assigned a unique welcome message to each Apache container.

```
None
sudo docker exec app1 sh -c 'echo "<h1>Welcome to Application
1</h1>" > /usr/local/apache2/htdocs/index.html'


sudo docker exec app2 sh -c 'echo "<h1>Welcome to Application
2</h1>" > /usr/local/apache2/htdocs/index.html'


sudo docker exec app3 sh -c 'echo "<h1>Welcome to Application
3</h1>" > /usr/local/apache2/htdocs/index.html'


sudo docker exec app4 sh -c 'echo "<h1>Welcome to Application
4</h1>" > /usr/local/apache2/htdocs/index.html'


sudo docker exec app5 sh -c 'echo "<h1>Welcome to Application
5</h1>" > /usr/local/apache2/htdocs/index.html'
```

4. Verify All Five Containers

```
None
sudo docker ps
```

Verified that all five containers were running with their respective port mappings.

Container	Host Port	Container Port
app1	3001	80
app2	3002	80
app3	3003	80
app4	3004	80
app5	3005	80

5. Configure Nginx Reverse Proxy

Opened the Nginx default configuration file.

```
None
sudo nano /etc/nginx/sites-available/default
```

Added the following reverse proxy configuration:

```
None
server {
    listen 80;

    location /app1/ {
        proxy_pass http://127.0.0.1:3001/;
    }
```

```
    location /app2/ {
        proxy_pass http://127.0.0.1:3002/;
    }

    location /app3/ {
        proxy_pass http://127.0.0.1:3003/;
    }

    location /app4/ {
        proxy_pass http://127.0.0.1:3004/;
    }

    location /app5/ {
        proxy_pass http://127.0.0.1:3005/;
    }
}
```

Saved the configuration using `Ctrl + O`, `Enter`, and `Ctrl + X`.

6. Test and Restart Nginx

Checked the Nginx configuration for syntax errors.

```
None
sudo nginx -t
```

Restarted the Nginx service.

```
None
sudo systemctl restart nginx
```

Checked the Nginx service status.

```
None
sudo systemctl status nginx
```

7. Verify All Five Applications Using Curl

Executed the following commands to verify that Nginx correctly forwarded requests to the respective backend containers.

```
None
curl http://localhost/app1/

curl http://localhost/app2/

curl http://localhost/app3/

curl http://localhost/app4/

curl http://localhost/app5/
```

8. Access Applications Through the Browser

Accessed each application using the EC2 Public IPv4 address.

Application	Browser URL

Application 1	`http://52.87.198.74/app1/`
Application 2	`http://52.87.198.74/app2/`
Application 3	`http://52.87.198.74/app3/`
Application 4	`http://52.87.198.74/app4/`
Application 5	`http://52.87.198.74/app5/`

Verify that the EC2 public IP is still current before using these URLs.

9. AWS Security Group Configuration

Configured the EC2 Security Group with the following inbound rule:

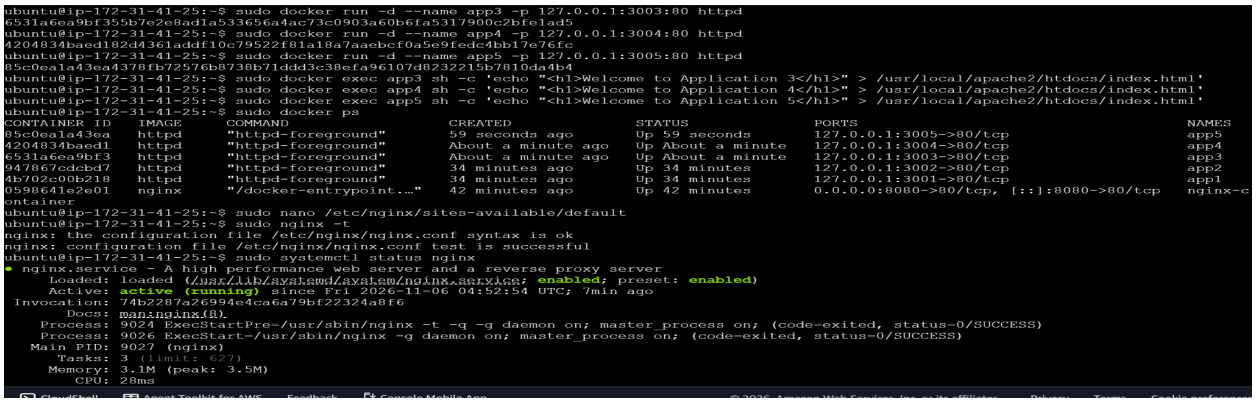
Type	Protocol	Port	Source
HTTP	TCP	80	0.0.0.0/0

Backend ports `3001–3005` were bound to localhost, so they did not need to be exposed publicly.

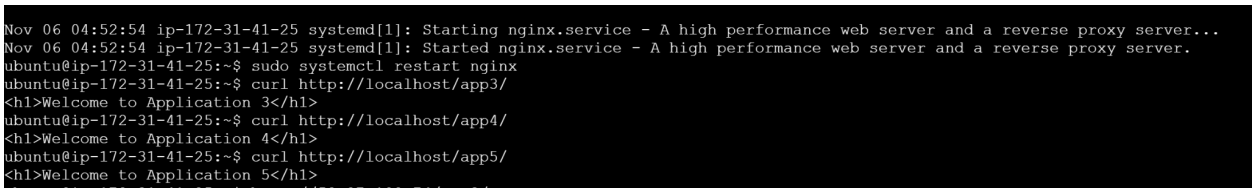
Description

Successfully implemented an Nginx reverse proxy for five Apache HTTPD Docker applications on an AWS EC2 Ubuntu instance. Initially, I checked the existing containers using `sudo docker ps` and created three additional containers (`app3`, `app4`, and `app5`) using `sudo docker run` commands with different localhost port mappings from `3003` to `3005`. Assigned separate welcome pages to all five applications using `sudo docker exec` commands. Verified the running containers and configured Nginx by editing `/etc/nginx/sites-available/default` and adding five separate location blocks with corresponding `proxy_pass` directives. Tested the configuration using `sudo nginx -t`, restarted the Nginx service, and checked its status. Finally, verified all five applications using `curl` commands and accessed them through different URL paths in the browser using the EC2 Public IPv4 address. Configured the AWS Security Group to allow inbound HTTP traffic on port `80`.

Result: Successfully deployed five Docker-based Apache applications and accessed all five through a single EC2 public IP using Nginx reverse proxy with different URL paths.



```
ubuntu@ip-172-31-41-25:~$ sudo docker run -d --name app3 -p 127.0.0.1:3003:80 httpd
6531a6ea9bf355b7e2e8ad1a533656a4ac73c0903a60b6fa5317900c2bfe1ad5
ubuntu@ip-172-31-41-25:~$ sudo docker run -d --name app4 -p 127.0.0.1:3004:80 httpd
4204834baed182d4361addf10c79522f81a18a7aaebcf0a5e9fedc4bb17e76fc
ubuntu@ip-172-31-41-25:~$ sudo docker run -d --name app5 -p 127.0.0.1:3005:80 httpd
85c0ea1a43ea4378fb72576b8730b71ddd3c38efa96107d8232215b7810da4b4
ubuntu@ip-172-31-41-25:~$ sudo docker exec app3 sh -c 'echo "<h1>Welcome to Application 3</h1>" > /usr/local/apache2/htdocs/index.html'
ubuntu@ip-172-31-41-25:~$ sudo docker exec app4 sh -c 'echo "<h1>Welcome to Application 4</h1>" > /usr/local/apache2/htdocs/index.html'
ubuntu@ip-172-31-41-25:~$ sudo docker exec app5 sh -c 'echo "<h1>Welcome to Application 5</h1>" > /usr/local/apache2/htdocs/index.html'
ubuntu@ip-172-31-41-25:~$ sudo docker ps
CONTAINER ID   IMAGE     COMMAND              CREATED              STATUS              PORTS                                   NAMES
85c0ea1a43ea   httpd     "httpd-foreground"   59 seconds ago       Up 59 seconds       127.0.0.1:3005->80/tcp                  app5
4204834baed1   httpd     "httpd-foreground"   About a minute ago   Up About a minute   127.0.0.1:3004->80/tcp                  app4
6531a6ea9bf3   httpd     "httpd-foreground"   About a minute ago   Up About a minute   127.0.0.1:3003->80/tcp                  app3
947867cdcbd7   httpd     "httpd-foreground"   34 minutes ago       Up 34 minutes       127.0.0.1:3002->80/tcp                  app2
4b702c00b218   httpd     "httpd-foreground"   34 minutes ago       Up 34 minutes       127.0.0.1:3001->80/tcp                  app1
059864e2e01    nginx     "/docker-entrypoint.…"   42 minutes ago   Up 42 minutes       0.0.0.0:8080->80/tcp, [::]:8080->80/tcp   nginx-c
ontainer
ubuntu@ip-172-31-41-25:~$ sudo nano /etc/nginx/sites-available/default
ubuntu@ip-172-31-41-25:~$ sudo nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
ubuntu@ip-172-31-41-25:~$ sudo systemctl status nginx
● nginx.service - A high performance web server and a reverse proxy server
     Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: enabled)
     Active: active (running) since Fri 2026-11-06 04:52:54 UTC; 7min ago
 Invocation: 74b2287a2694e4ca6a79bf22324a8f6
       Docs: man:nginx(8)
    Process: 9024 ExecStartPre=/usr/sbin/nginx -t -q -g daemon on; master_process on; (code=exited, status=0/SUCCESS)
    Process: 9026 ExecStart=/usr/sbin/nginx -g daemon on; master_process on; (code=exited, status=0/SUCCESS)
   Main PID: 9027 (nginx)
      Tasks: 3 (limit: 627)
     Memory: 3.1M (peak: 3.5M)
        CPU: 28ms
```



```
Nov 06 04:52:54 ip-172-31-41-25 systemd[1]: Starting nginx.service - A high performance web server and a reverse proxy server...
Nov 06 04:52:54 ip-172-31-41-25 systemd[1]: Started nginx.service - A high performance web server and a reverse proxy server.
ubuntu@ip-172-31-41-25:~$ sudo systemctl restart nginx
ubuntu@ip-172-31-41-25:~$ curl http://localhost/app3/
<h1>Welcome to Application 3</h1>
ubuntu@ip-172-31-41-25:~$ curl http://localhost/app4/
<h1>Welcome to Application 4</h1>
ubuntu@ip-172-31-41-25:~$ curl http://localhost/app5/
<h1>Welcome to Application 5</h1>
```

Application 3

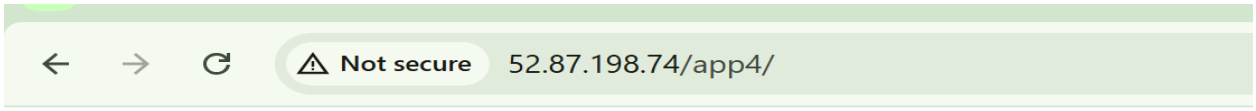
`http://52.87.198.74/app3/`



Welcome to Application 3

Application 4

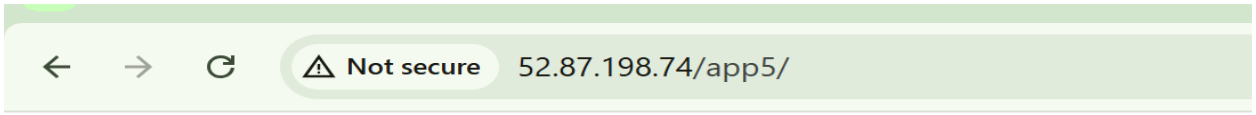
http://52.87.198.74/app4/



Welcome to Application 4

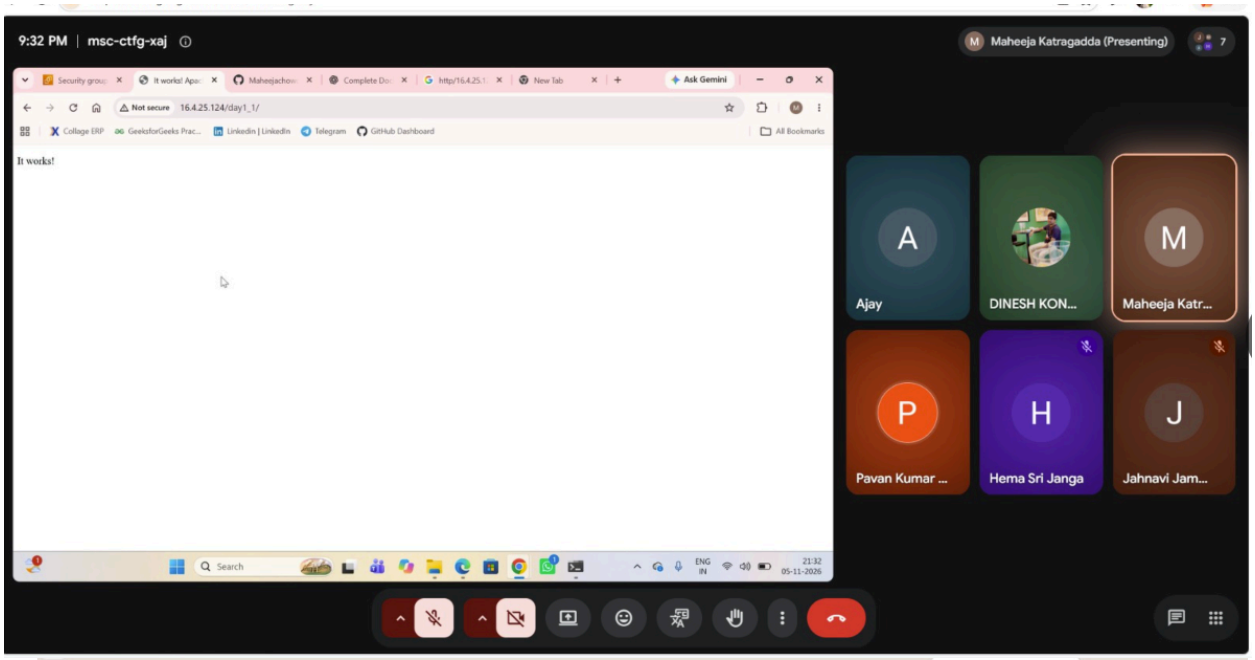
Application 5

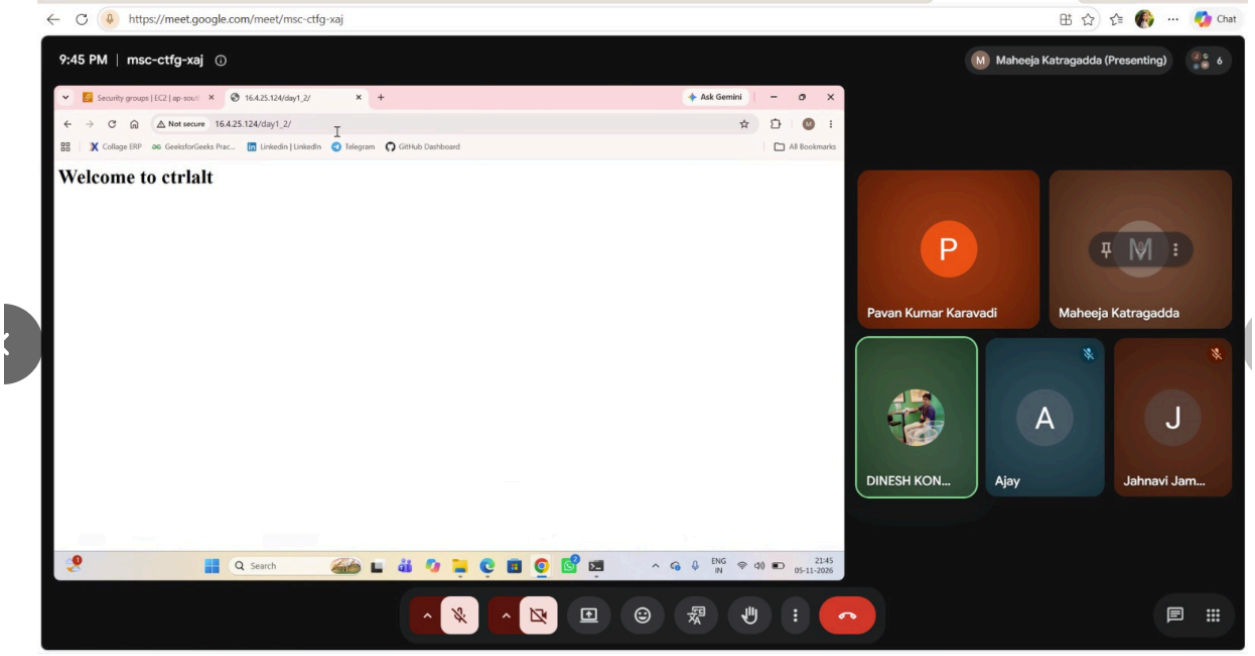
http://52.87.198.74/app5/



Welcome to Application 5

The assigned tasks were successfully completed by the team members yesterday as scheduled. However, due to some internal issues, the Scriber (Hema Sri) was unable to capture the screenshots during the execution. Therefore, the same tasks were performed again today, and the required screenshots were captured and added to the document for proper documentation and verification.





During this session, we worked on deploying and managing multiple web applications using AWS EC2, Docker, Apache HTTPD and Nginx. We started by connecting to an Ubuntu EC2 instance and learning how to manage the server through the terminal. We updated the Ubuntu package repositories and installed Nginx. We verified the installation by checking the Nginx version, service status and listening ports.

Next, we installed and explored Docker. We learned how to download Docker images, create containers, run containers in detached mode and map container ports to host ports. We also understood how to execute commands inside running containers using `docker exec`. We deployed an Nginx Docker container and accessed its default welcome page through the EC2 public IP address using port `8080`.

After that, we worked on the main assignment: Configuring Nginx as a Reverse Proxy for Five Apache HTTPD Applications. We created five Apache HTTPD containers (`app1` to `app5`) and mapped them to different localhost ports ranging from `3001` to `3005`. We assigned a separate welcome page to each container using Docker commands.

We then configured the host Nginx server as a reverse proxy by adding five location blocks in the Nginx default configuration file. Each URL path (`/app1/` to `/app5/`) was mapped to its corresponding Apache backend container using the `proxy_pass` directive. We tested the configuration using `sudo nginx -t`, restarted the Nginx service and verified the applications using `curl` commands. Finally, we accessed all five applications through a single EC2 public IP address using different URL paths.

We also explored troubleshooting procedures, including checking Nginx service status, examining error logs, verifying port usage and identifying configuration issues.

Final Outcome

Successfully deployed five Apache HTTPD applications inside Docker containers on an AWS EC2 Ubuntu instance and configured Nginx as a reverse proxy to access all five applications through a single public IP address using different URL paths.

Team Roles and Contributions

1. Dinesh Konda (Lead): Guided the team and supported the execution of the assigned tasks.
2. Katragadda Maheeja (Driver): Shared her screen and performed the commands during the practical session.
3. Jampani Jahnavi (Navigator): Rechecked the commands and verified the execution steps.
4. Karavadi Pavan Kumar (Breaker): Guided Maheeja during command execution and helped identify potential issues.
5. Hema Sri Janga (Scribe): Prepared the documentation and organized the screenshots of the practical work.
6. Ajay Siddela (Presenter): Presented the team's work and explained the completed tasks.

10. GitHub Submission

All Docker commands, implementation steps, configuration details, screenshots, troubleshooting activities and learning outcomes are maintained in the team's GitHub repository.

GitHub Repository: https://github.com/Maheejachowdary/Ctrl-Alt-Deploy

Folder: day-01/

Submission Date: 06 November 2026

Submitted By: Hema Sri Janga (Scribe)

Team Name: Ctrl+Alt+Deploy

**End of Day 1 Session Report